

Assignment 3 - Customer Service Data Analyst Agent

Due date: 29.5.26

Intro

In this assignment you'll build a data analyst agent for the **Bitext Customer Service** dataset. The agent will answer user questions about the data. The questions can be either structured ("How many refund requests?") or open-ended ("Summarize the FEEDBACK category"). Out-of-scope questions should be gracefully declined.

You'll implement the agent as a LangGraph ReAct graph with persistent memory, and expose its tools via a FastMCP server.

General Instructions

1. **Submission:** Submit a GitHub repository (link or zip). Make sure the repo is accessible to the graders. Include the first and last names of both students in the repo name.
2. **Dependencies:** The repo must include a [requirements.txt](#) or [pyproject.toml](#) with version numbers.
3. **README:** Include a [README.md](#). A grader should be able to clone your repo and have the agent running within 5 minutes by following your README alone. At minimum, cover: setup steps, how to run the CLI, how to connect a client to one of its tools with MCP and a brief architecture overview (what model you chose and why, what tools you defined).
4. **Model choice:** State which Nebius Token Factory model(s) you're using and briefly justify the choice in your README. If you use different models for different roles (e.g., a smaller model for routing, a larger one for generation), explain that. As usual, only Nebius Token Factory models are allowed for LLM calls.
5. **Code quality:** Use meaningful variable/function names, type hints, and docstrings on public functions.
6. **Debugging tip:** Use LangGraph Studio to visually trace and debug your agent's graph. It's free and purpose-built for LangGraph. Alternatively, enable verbose logging in your agent to print every tool call, observation, and reasoning step to the console.

The Dataset

Bitext - Customer Service Tagged Training Dataset ([link](#))

Bitext is a synthetic dataset of customer support queries paired with agent responses. Its original purpose is models fine-tuning, but here we use it as a data source for our agent to analyze.

Familiarize yourself with the data and its features before you start building.

Task 1 - Build the Initial Agent (50 pts)

Build a LangGraph-based ReAct agent that answers user questions about the dataset. The agent should support three types of queries:

Structured - questions with concrete, data-driven answers:

- "What categories exist in the dataset?"
- "How many refund requests did we get?"
- "Show me 3 examples from the SHIPPING intent."
- "What is the distribution of intents in the ACCOUNT category?"

Unstructured - open-ended questions requiring summarization:

- "Summarize the FEEDBACK category."
- "How do customer service representatives typically respond to cancellation requests?"

Out-of-scope - questions unrelated to the dataset:

- "Who won the 2024 Champions League?"
- "Write me a poem about customer service."

Detailed requirements appear in the next page.

Grading breakdown: query router (15), tools with clear descriptions and Pydantic schemas (15), multi-step reasoning (10), CLI with reasoning output (5), max iterations fallback (5).



Requirements

1. **Max iterations:** Set a maximum iteration limit on your agent loop (A value between 10–15 is a reasonable starting point). If the agent hasn't produced a final answer after that number of iterations, it should return a graceful fallback message rather than spinning forever.
2. **Query router:** Implement a dedicated router node that classifies the incoming query as structured, unstructured, or out-of-scope before the agent begins tool selection. Out-of-scope queries should be declined politely - the agent should not answer them from the LLM's general knowledge.
3. **Tools:** Define a set of tools the agent can use. Each tool must have a clear name, description, and Pydantic input schema defining its parameters. Typing your return values is good practice too.

Remember: Your tool descriptions are as important as the tool logic. If a human can't tell when to use a tool from its description alone, neither can the LLM. “*A few well-designed tools beat many poorly described ones*” (T. Braude).

4. **Multi-step reasoning:** The agent must handle queries that require chaining multiple tools. For example:
 - "How many refund requests did we get?" →
`filter_by_intent("get_refund") → count_rows()`
5. **CLI interface:** The agent should run from the command line (e.g., `python main.py`) and drop into an interactive conversation loop. Print the agent's reasoning steps (tool calls and observations), not just the final answer.

Examples for queries you can test your agent on:

- "What categories exist in the dataset?"
- "How many refund requests did we get?"
- "Show me 5 examples of the SHIPPING category."
- "Summarize how agents respond to complaint intents."
- "Show me examples of people wanting their money back."
- "What is the distribution of intents in the ACCOUNT category?"
- "What's the best CRM software for handling complaints?" (out-of-scope)
- "Who is the president of France?" (out-of-scope)

Think about what purpose each of the above tests serve.



Task 2 - Memory (30 pts)

2a. Conversation Memory / Episodic (20 pts)

Enable the agent to remember conversation history across turns and across restarts.

1. Use LangGraph **checkpoints** to persist conversation state.
2. Support a **session ID** argument (e.g., `python main.py --session my_session`). The same session ID should restore the same conversation even after restarting the app.
3. The agent must handle follow-up queries that reference earlier turns:
 - "Show me 3 examples from the REFUND category" → *[agent shows examples]* → "Show me 3 more"
 - "How many complaints did we get?" → "What about refunds?" → "What is the total count of the last two?"

[This](#) notebook demonstrates the checkpointer concept using in-memory storage. You can use it as reference, but note that *MemorySaver* is in-memory only and won't survive a restart. Look into *SqliteSaver* or *PostgresSaver* for persistence.

2b. User Profile (10 pts)

Enable the agent to build and maintain a persistent profile for each user, stored separately from the conversation history:

1. The profile should capture distilled facts such as the user's name, topics they frequently ask about or preferences. It's **not** a replay of past messages.
2. It should be able to answer questions like "What do you remember about me?" by referencing this profile, and should naturally update it as new information emerges in conversation.
3. It must persist across restarts, whether through the checkpointer, a per-user file, or another approach.

You can implement this using a summary node in the graph, a per-user context file (context.md) or other approaches. [Here](#) you can find more ideas.



Task 3 - MCP Server (20 pts)

Build an MCP server using [FastMCP](#). Expose at least 3 of your tools as MCP tools. In

your README part that shows how to start the server, include a short section showing how to connect a client to call one of its tools.

Bonus A - Streamlit UI (+10 pts)

Wrap your CLI agent in a **Streamlit chat app**.

1. The user types questions in a chat interface and sees the agent's responses.
2. Display the agent's reasoning steps (tool calls and results), not just the final answer.
3. Add a **session ID** input in the sidebar so users can switch between or resume conversations.

Bonus B - Query Recommender (+10 pts)

Add an interactive query recommendation feature. When the user asks **"What should I query next?"** (or similar), the agent should:

1. Look at the conversation history (episodic memory) and user profile.
2. Suggest a relevant follow-up query but don't execute it immediately.
3. Let the user refine the suggestion through conversation.
4. Only execute the query once the user confirms.

Example Flow

User: "What should I query next?"

Agent: "Based on your interest in refund data, you might want to see the distribution of intents in the REFUND category."

User: "I'd rather see examples instead."

Agent: "Then I'd suggest: show 5 examples from the REFUND category. Should I go ahead?"

User: "Yes, do it."

Agent: [executes `show_examples(n=5, category='REFUND')` and displays results]